

EEE283

Digital Logic Design

Course Instructor: A K M Anindya Alam

Topic 3:

Fundamental Logic Gates

Analog vs. Digital

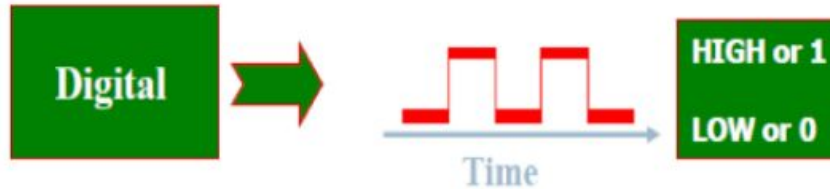
Analog signal- one whose output varies **continuously** in step with the input.

Example:



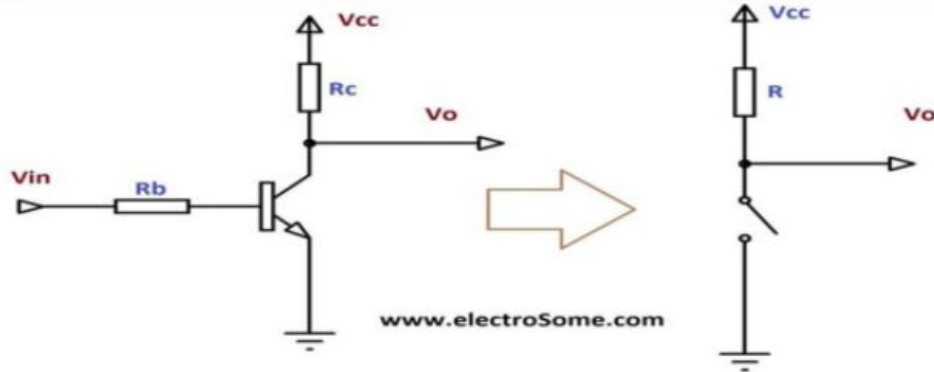
Digital signal- one whose output varies at discrete voltage levels commonly called HIGH or LOW (1 or 0).

Example:

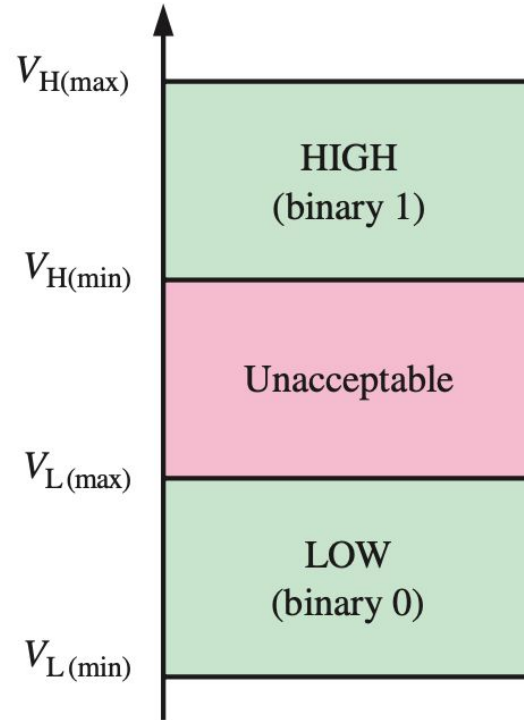


Digital (Boolean) Logic

- At any point in circuit, only two Voltage states are present – HIGH and LOW
- Also sometimes called TRUE or FALSE. In Boolean logic, 0 and 1
- Generally, +5V is considered high, 0V considered ground



Digital (Boolean) Logic : Binarization



Digital Logic

- Digital Signals are used to denote **TRUE** (1) and **FALSE** (0) conditions.
- We can interpret logical arguments using digital logic.

For example:

Suppose, you are playing FIFA on your PS4. *For a finesse shot, you need to press **R1** and  simultaneously.*

This can be interpreted using digital logic.

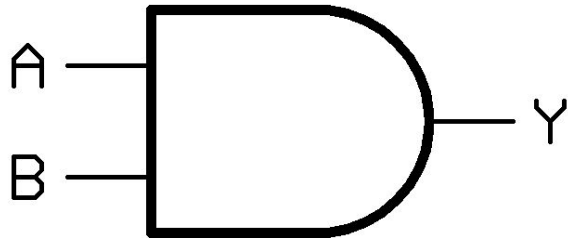
R1 (X)	 (Y)	Finesse Shot (Output Y)
TRUE (1)	TRUE (1)	TRUE (1)
TRUE (1)	FALSE (0)	FALSE (0)

Fundamental Logic Gates

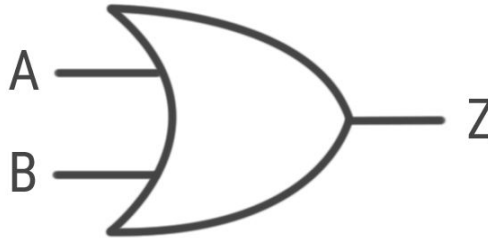
- For arithmetic operations, we need operators (+, -, $\times$, $\div$)
- For logic arguments, we also need operators. **These are called Logic Gates.** We can also call them logical operators.

There are many such **Logic Gates**. But there are three **fundamental logic gates**.

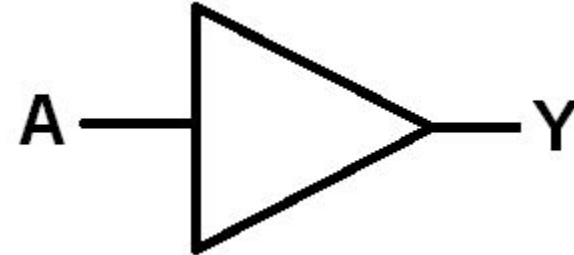
◦ AND



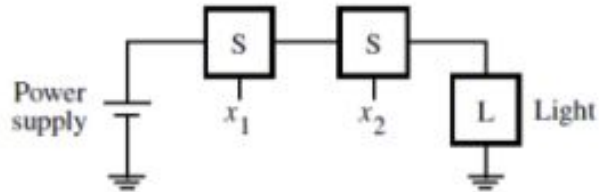
◦ OR



◦ NOT



Logical AND Operation



(a) The logical AND function (series connection)

Operation: $A \cdot B$

Symbol

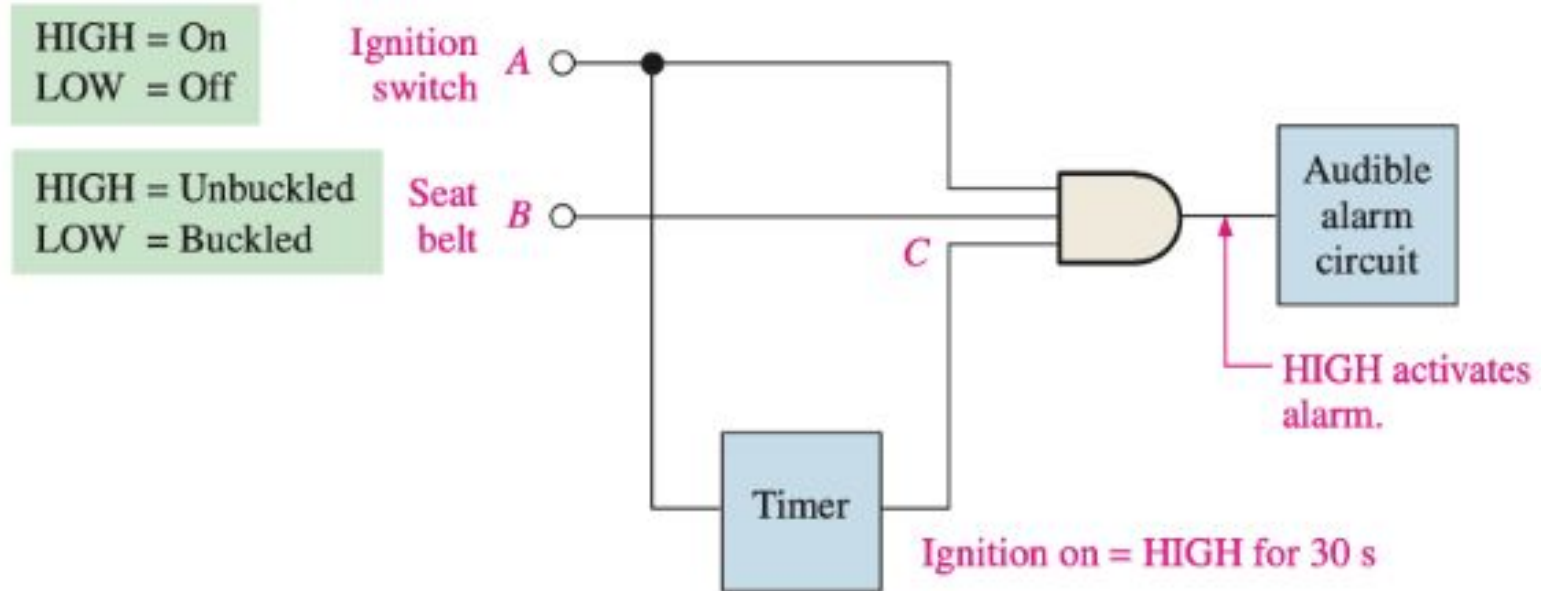


Truth table for a 2-input AND gate.

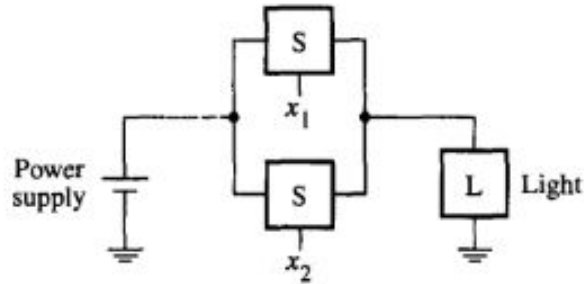
Inputs		Output
A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

1 = HIGH, 0 = LOW

Application of AND Logic

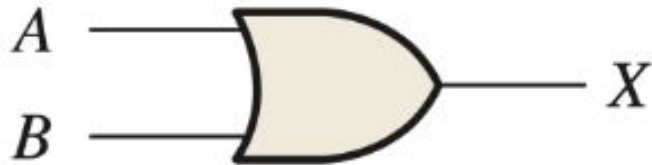


Logical OR Operation



Operation: $A+B$

Symbol



Truth table for a 2-input OR gate.

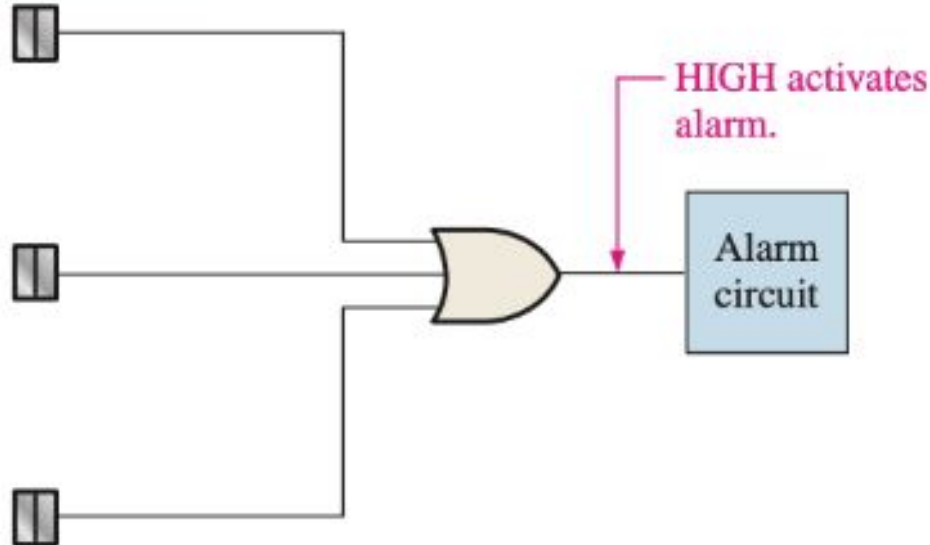
Inputs		Output
<i>A</i>	<i>B</i>	<i>X</i>
0	0	0
0	1	1
1	0	1
1	1	1

1 = HIGH, 0 = LOW

Application of OR Logic

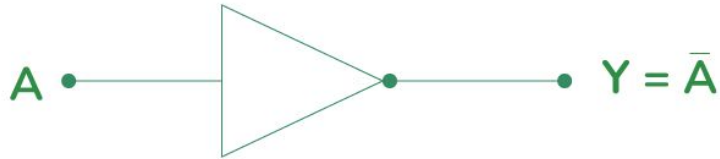
Open door/window
sensors

HIGH = Open
LOW = Closed



Logical NOT Operation and Application

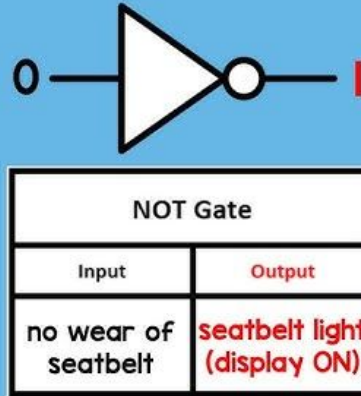
NOT Gate



Truth Table

A (Input)	$Y = \bar{A}$ (Output)
0	1
1	0

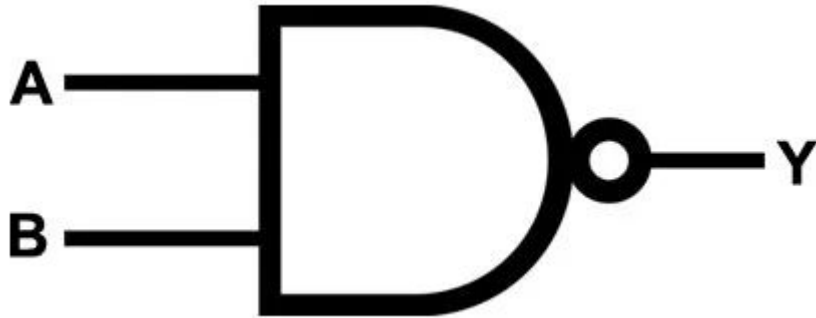
NOT Gate



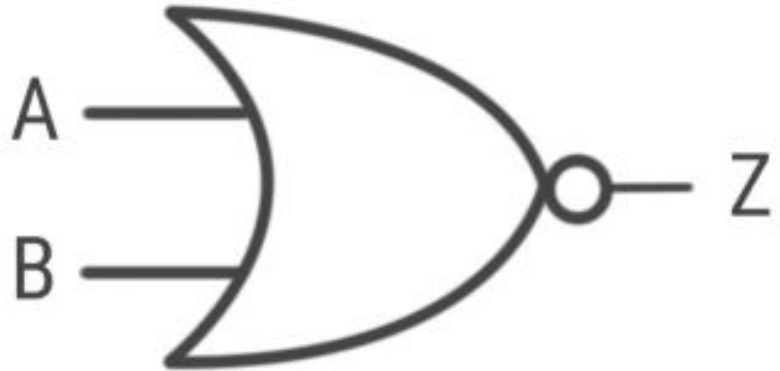
Universal Logic Gates

There are two special logic gates. These are called **Universal Logic Gates**. This is because, in real life, they are used to implement **Logic**, including the fundamental ones. **Fabrication is cost-effective.**

◦ NAND



◦ NOR



Logical NAND Operation

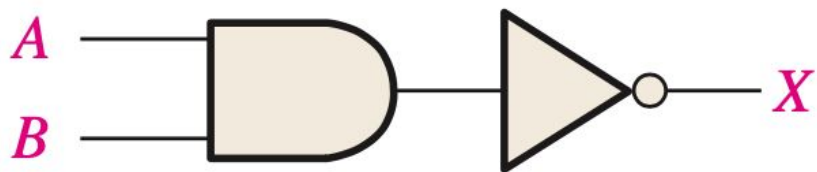
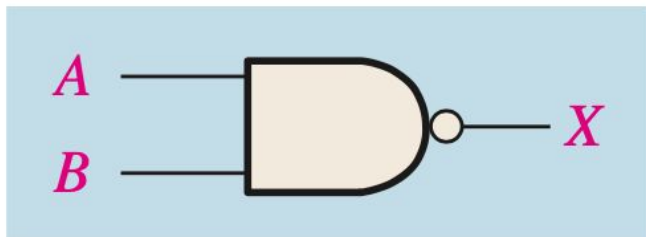
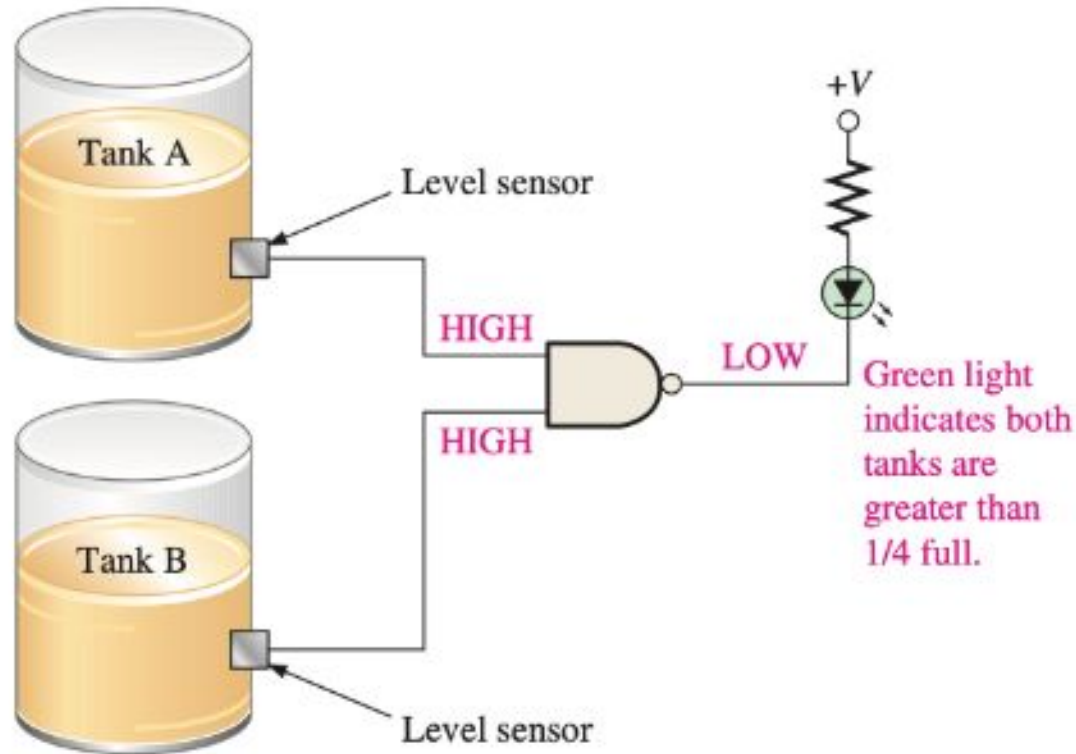


TABLE 3-8

<i>A</i>	<i>B</i>	$\overline{AB} = X$
0	0	$\overline{0 \cdot 0} = \overline{0} = 1$
0	1	$\overline{0 \cdot 1} = \overline{0} = 1$
1	0	$\overline{1 \cdot 0} = \overline{0} = 1$
1	1	$\overline{1 \cdot 1} = \overline{1} = 0$

Application of NAND Logic



Logical NOR Operation

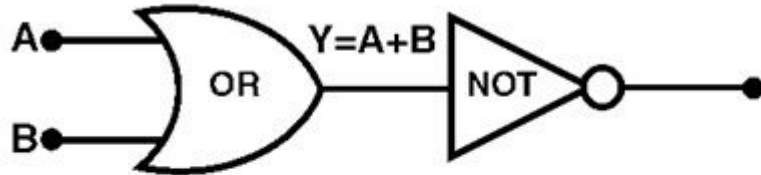
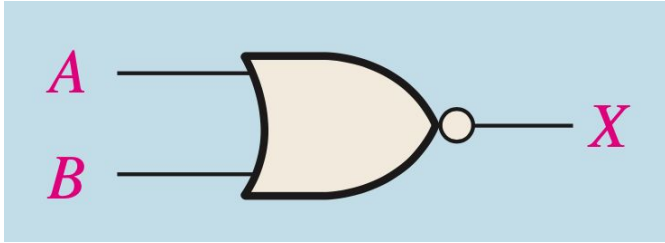
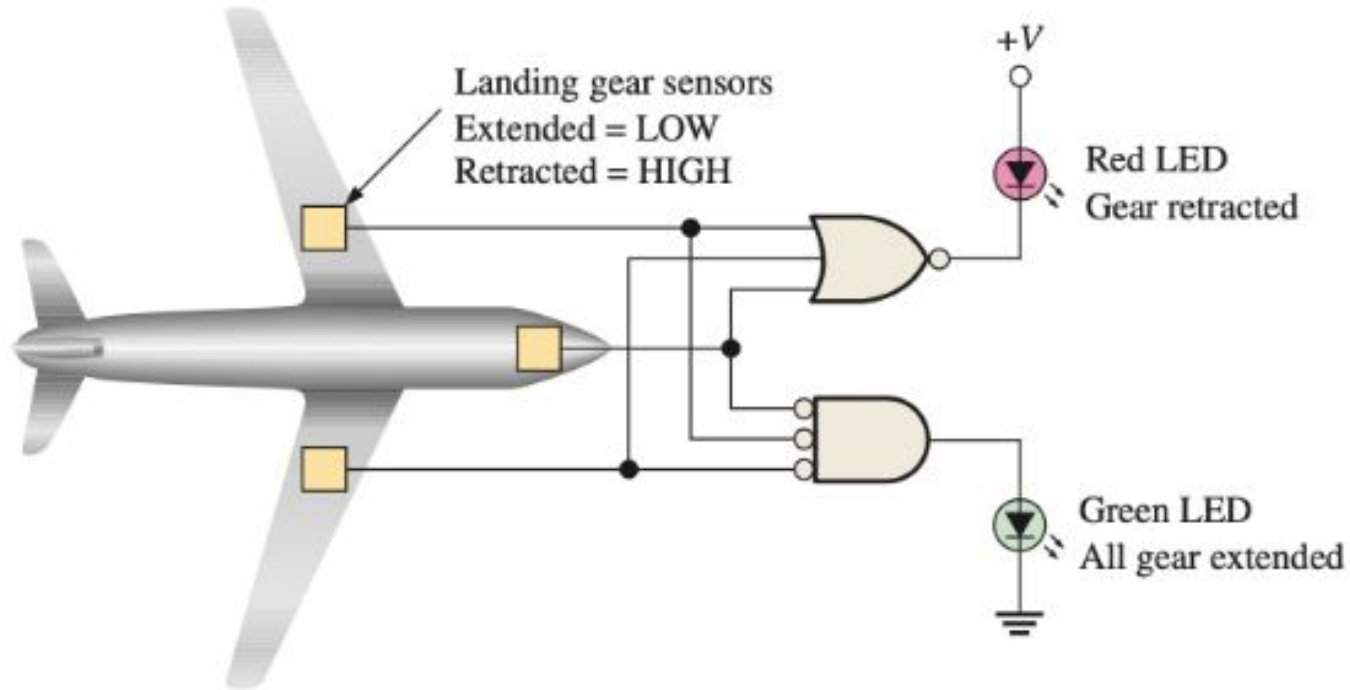


TABLE 3-10

<i>A</i>	<i>B</i>	$\overline{A + B} = X$
0	0	$\overline{0 + 0} = \overline{0} = 1$
0	1	$\overline{0 + 1} = \overline{1} = 0$
1	0	$\overline{1 + 0} = \overline{1} = 0$
1	1	$\overline{1 + 1} = \overline{1} = 0$

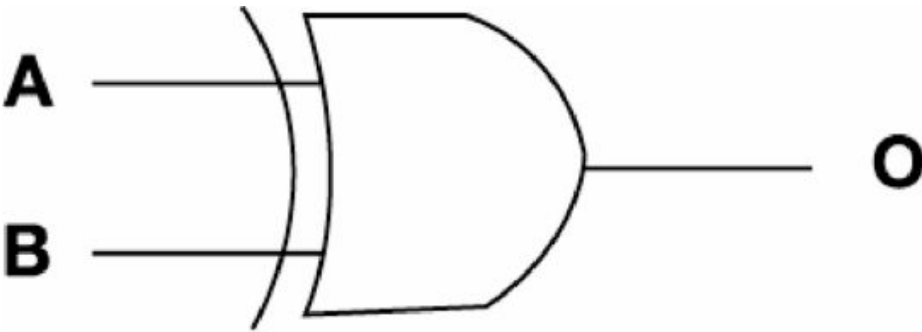
Application of NOR Logic



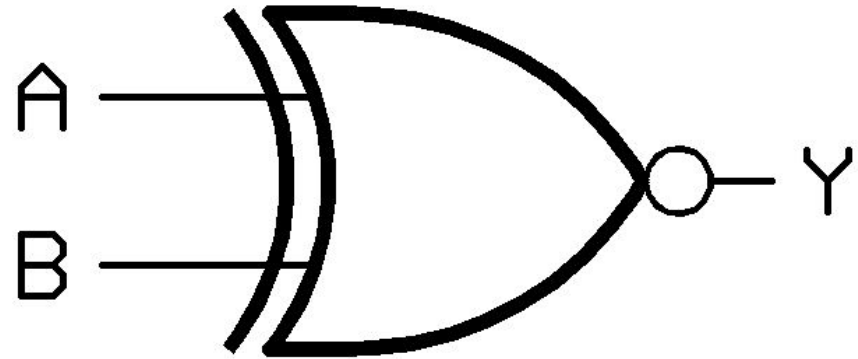
Exclusive Logic Gates

These are two complex logic gates for special functions. These are called **Exclusive Logic Gates**.

◦ **XOR**



◦ **XNOR**



Logical XOR

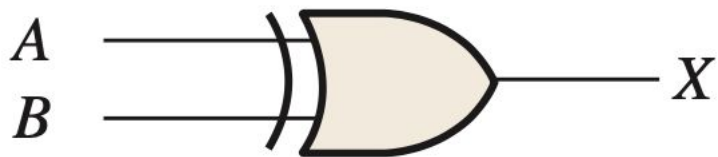


TABLE 3-11

Truth table for an exclusive-OR gate.

Inputs		Output
A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

Logical XNOR

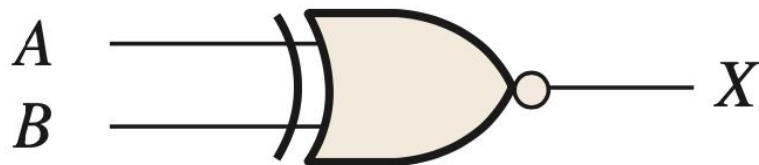


TABLE 3-12

Truth table for an exclusive-NOR gate.

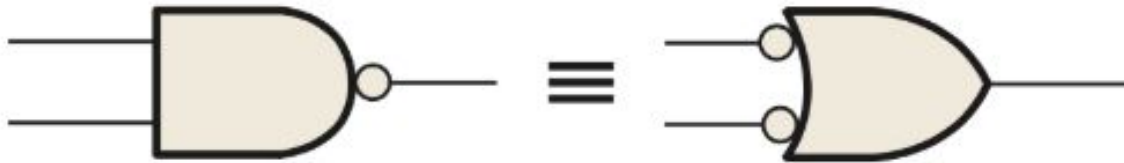
Inputs		Output
A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

Alternate Logic Gates



NOR

Negative-AND



NAND

Negative-OR

Alternate Logic Gates

Original Gates	Alternate Gates
	
	
	

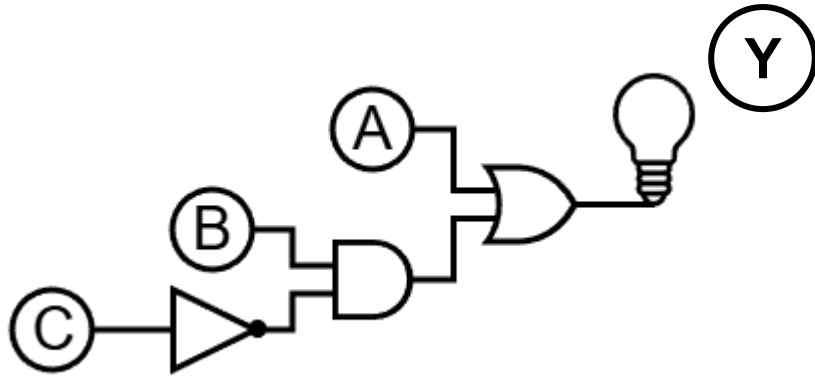
This will become easier to understand once we explore boolean algebra

Boolean Expression of Logic Gates

Logic Gate	Symbol	Description	Boolean
AND		Output is at logic 1 when, and only when all its inputs are at logic 1, otherwise the output is at logic 0.	$X = A \cdot B$
OR		Output is at logic 1 when one or more are at logic 1. If all inputs are at logic 0, output is at logic 0.	$X = A + B$
NAND		Output is at logic 0 when, and only when all its inputs are at logic 1, otherwise the output is at logic 1	$X = \overline{A \cdot B}$
NOR		Output is at logic 0 when one or more of its inputs are at logic 1. If all the inputs are at logic 0, the output is at logic 1.	$X = \overline{A + B}$
XOR		Output is at logic 1 when one and Only one of its inputs is at logic 1. Otherwise is it logic 0.	$X = A \oplus B$
XNOR		Output is at logic 0 when one and only one of its inputs is at logic 1. Otherwise it is logic 1. Similar to XOR but inverted.	$X = \overline{A \oplus B}$
NOT		Output is at logic 0 when its only input is at logic 1, and at logic 1 when its only input is at logic 0. That's why it is called and INVERTER	$X = \overline{A}$

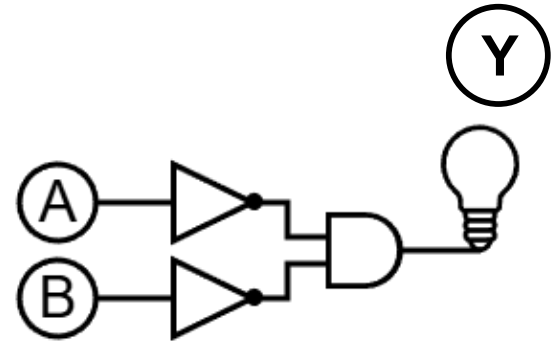
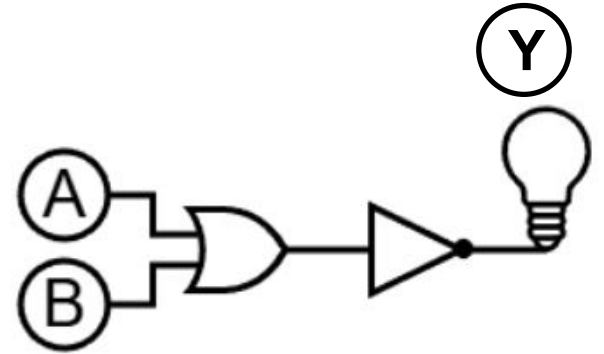
Complex Logic Implementation

$$Y = A + B\bar{C}$$



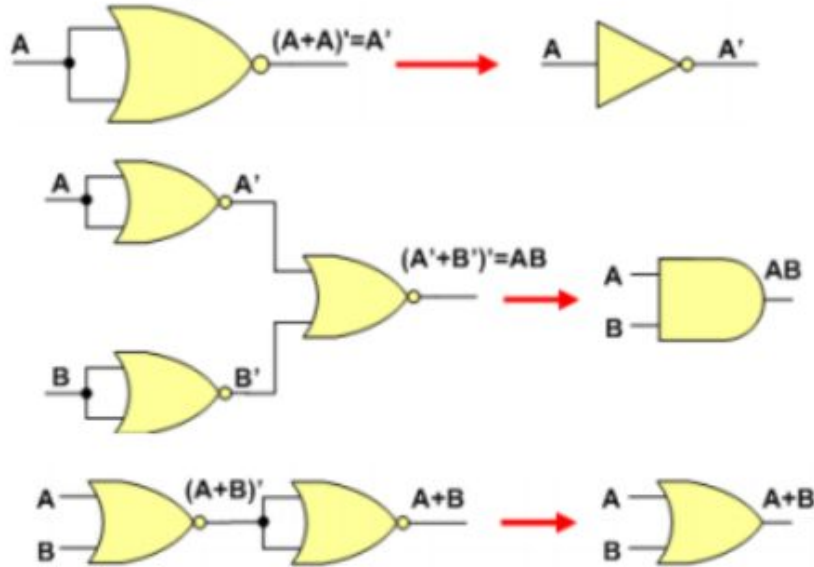
Find the truth table for this logic.

$$Y = \overline{A + B}$$

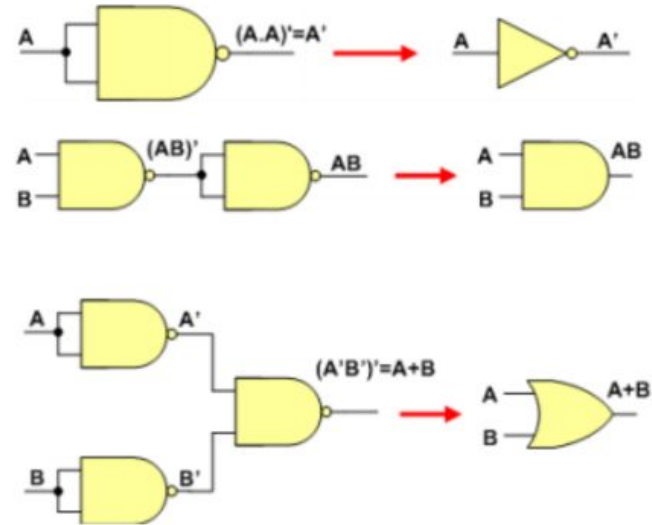


Representation of Logic Gates using Universal Gates

NOR Gate Implementation:



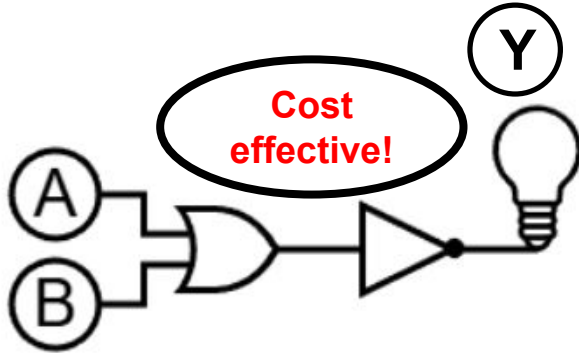
NAND Gate Implementation:



Cost of Logic Circuit

The cost of logic implementation is the number of input terminals at each logic gates. Each input is directly related to fabricating a **CMOS pair**, adding to the cost of implementation of the circuit. **Our objective is to design our logic circuit in a way so we can minimize the cost.**

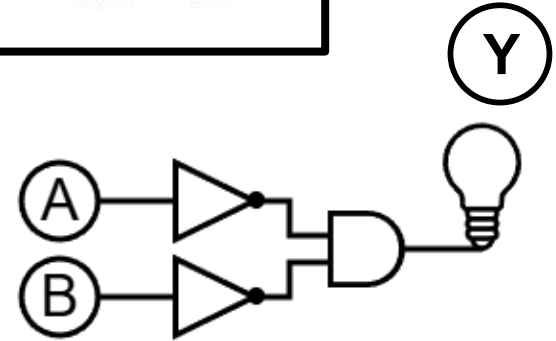
$$Y = \overline{A + B} = \overline{A} \cdot \overline{B}$$



NOT gate: 1 input

OR Gate: 2 inputs

Cost: 1 input NOT(x1) + 2 input OR(x1) = 3



NOT gate: 1 input

OR Gate: 2 inputs

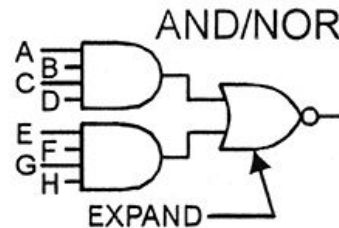
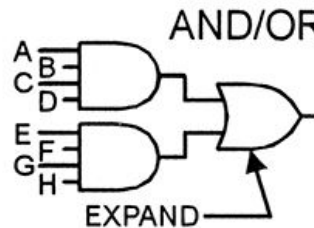
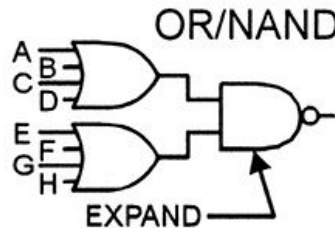
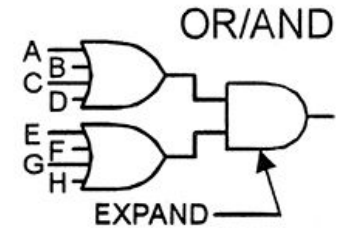
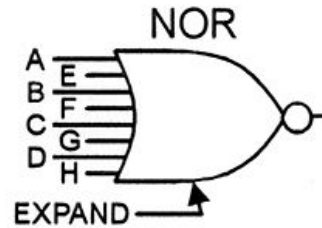
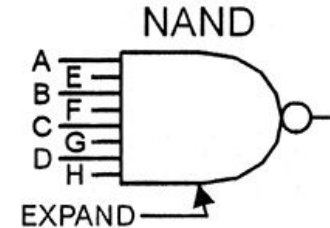
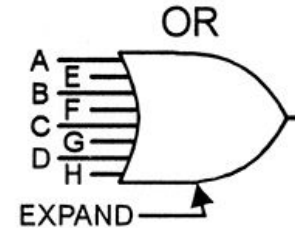
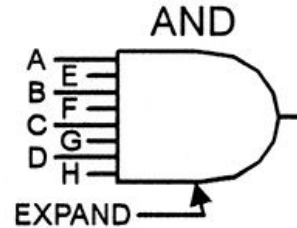
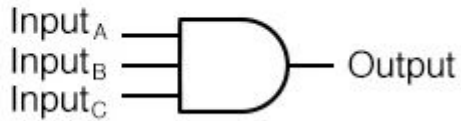
Cost: 1 input NOT(x2) + 2 input OR(x1) = 4

Multi-input Logic Gates

2 - input AND gate



3 - input AND gate



It is possible to combine multiple 2-input logic gates to build multi-logic gates but we can also fabricate multi-input gates which is preferred. **Can you explain why?**